Guardian Chip: A Compact NFC Access-Control ASIC with Mutual AES-128 Authentication

PHILIPP SEELOS, Hochschule RheinMain, Germany

This report documents the design and implementation of the *Guardian Chip*, a custom application-specific integrated circuit (ASIC) developed for the LAYR Open Chip Challenge 2025/26 [14]. The chip implements a hardware door-lock controller that authenticates contactless smart cards using the LAYR Level 1 Authenticated Identification Protocol, a symmetric challenge-response protocol based on AES-128.

The design is described in Verilog/SystemVerilog and targets the IHP SG13G2 130 nm BiCMOS open-source process [9], to be fabricated as a QFN-24 package via the LibreLane open-source ASIC flow [11]. The chip communicates with an NXP MFRC522 NFC reader [17] and a Microchip AT25010B SPI EEPROM [15] over a shared 4 MHz SPI bus. The authentication logic performs a mutual challenge-response handshake, derives an ephemeral session key, decrypts the card identity, and checks the result against a six-entry whitelist stored in non-volatile memory. AES-128 is realised as a compact iterative engine that processes one round per clock cycle and uses a compact composite-field S-box [2, 3] to minimise silicon area. The mathematical foundation for this S-box was established by Canright [2]; the Verilog implementation used in this work was published by GitHub user *coorus* [3]. Beyond the standard LAYR Level 1 protocol, a key-rollover mechanism was designed and implemented that allows the pre-shared key to be updated over the air as part of a successfully authenticated transaction. To allow seamless software-level testing without physical JavaCards, a companion Android Host Card Emulation (HCE) application that fully implements the card side of the protocol was also developed.

Additional Key Words and Phrases: ASIC, NFC, AES-128, challenge-response authentication, SPI, EEPROM, open-source PDK, access control, LAYR

1 Introduction

Physical access control is a fundamental problem in security engineering. Traditional keycard systems either store the permitted identifiers in plaintext and send them unencrypted over the wireless link—making replay attacks trivial [8]—or rely on proprietary, poorly documented protocols whose security is hard to reason about.

The LAYR Open Chip Challenge [14] poses exactly this problem: design a microchip that controls a 12 V door lock, reads a contactless smart card via ISO 14443-A/4 [12, 13], and decides whether to grant access. The challenge defines four security levels; this work targets *Level 1*, which mandates the *LAYR Authenticated Identification Protocol*: a symmetric, AES-based mutual challenge-response scheme that provides both card-to-terminal and terminal-to-card authentication and establishes a fresh ephemeral session key for every transaction.

Beyond the mandatory protocol requirements, a key-rollover mechanism was developed as an additional security feature that enables the pre-shared key to be updated over the air as part of an authenticated transaction, without requiring physical EEPROM access.

Rather than targeting an FPGA prototype only, the goal is to produce a tape-out-ready ASIC. The chip is designed with the IHP SG13G2 130 nm open-source PDK, synthesised and placed-and-routed with the LibreLane flow, and packaged in a QFN-24 footprint as specified by the challenge.

The remainder of this report is organised as follows. Section 2 reviews related work on compact hardware AES and NFC security. Section 3 describes the system architecture. Section 4 analyses the security of the implemented protocol. Section 5 details each RTL module. Section 6 presents the Android companion application. Section 7 covers the physical

This work was carried out as part of the course *Advanced IC Design* at Hochschule RheinMain, supervised by Prof. Dr. Steffen Reith.
Author's Contact Information: Philipp Seelos, Hochschule RheinMain, Germany, philipp.seelos@student.hs-rm.de.

design. Section 8 outlines the development process. Section 9 describes the verification strategy. Section 10 discusses limitations and future work, and Section 11 concludes the report.

2 Related Work

Compact hardware AES.. Implementing AES in hardware with a small footprint is well studied. Satoh et al. [20] showed that a composite-field representation of GF(2^8) over GF(2^4) allows the S-box to be realised with significantly fewer gates than a lookup-table approach. Canright [2] refined this into an extremely compact algebraic construction. A Verilog implementation of this design was published by GitHub user *coorus* [3] and is adopted in this work. Feldhofer et al. [7] demonstrated that AES is practical for RFID devices with strong resource constraints.

NFC and RFID security. Hancke and Kuhn [8] formalised relay attacks against contactless cards, motivating the need for challenge-response schemes that cannot be relayed without increasing round-trip latency. The LAYR protocol addresses this at the application layer. Host Card Emulation (HCE) on Android, introduced in Android 4.4, enables a smartphone to emulate an ISO 14443-4 card without requiring a hardware Secure Element, which is exploited for the companion simulator in Section 6.

Challenge-response authentication. The protocol structure—each party encrypts a nonce from the other party and returns the result—is a well-known symmetric authentication pattern [4]. The specific instantiation used by LAYR derives a fresh ephemeral key from the exchanged nonces, similar in spirit to the key derivation used in EMV contactless [6].

3 System Architecture

3.1 Overview

Figure 1 shows the top-level architecture. The chip integrates five main RTL subsystems around a shared SPI bus:

(1) **SPI Master** – a byte-oriented SPI Mode 0 master clocked at 4 MHz, shared by both the NFC reader and the EEPROM.
(2) **NFC Interface** (`mfrc522_apdu_interface`) – drives the MFRC522 reader, performs ISO 14443-A card detection, brings up an ISO 14443-4 link (RATS / ATS exchange), and provides a byte-stream APDU interface to the LAYR authentication core.
(3) **LAYR Core** (`layr_core`) – implements the complete state machine for the LAYR Level 1 protocol, including a key-rollover extension designed beyond the standard LAYR specification.
(4) **AES Core** (`aes_core` / `aes_iterative`) – a compact iterative AES-128 engine used for all encryption and decryption operations in the authentication core.
(5) **EEPROM Interface** (`at25010_interface`) – reads the pre-shared key, the nonce counter, and the ID whitelist from the AT25010B on power-up, and persists updated values when required.

3.2 Pin Interface and Packaging

The chip is packaged in a QFN-24 footprint as required by the LAYR challenge specification [14]. Table 1 lists the externally used pins. The remaining pins carry power and ground rails. Five bidirectional user-IO pads are present in the pad ring but are currently held as inputs (unused) and reserved for future use.

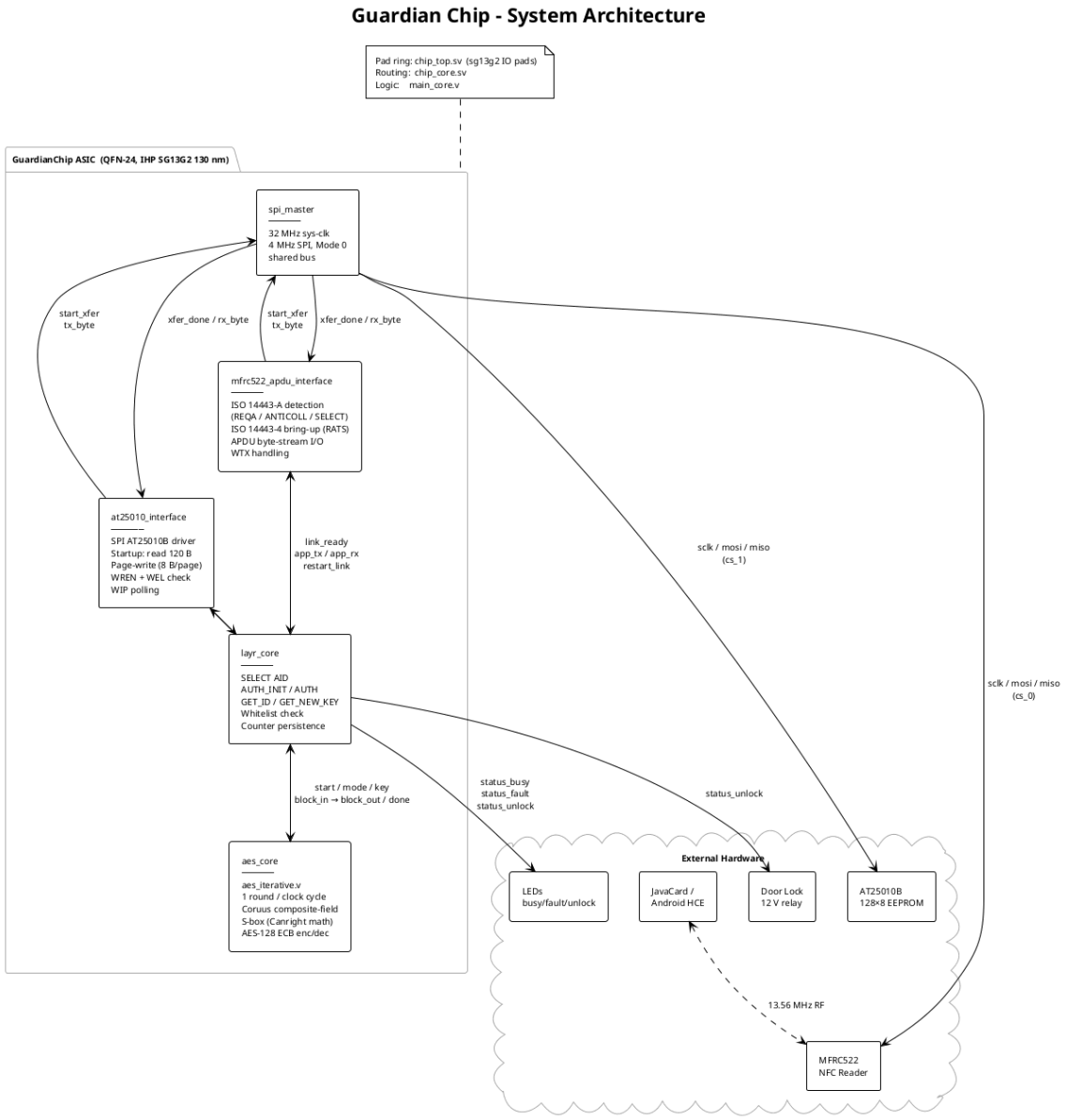

Fig. 1. System architecture of the Guardian Chip. Arrows denote the direction of primary data flow. The SPI bus (sclk, mosi, miso) is shared; CS lines select the active peripheral.

3.3 Module Hierarchy

`chip_top.sv` Instantiates all IHP SG13G2 pad cells: power pads (`IOPadVdd`, `IOPadVss`, `IOPadIOVdd`, `IOPadIOVss`), unidirectional input/output pads, and bidirectional tri-state pads. Provides the polarity inversion for the active-low internal reset from the active-high external pin.

Table 1. Externally used signal pins (QFN-24).

Pin	Name	Direction	Function
1	`rst`	Input	Active-high synchronous reset
2	`sys_clk`	Input	System clock (32 MHz)
13	`cs_1`	Output	AT25010B SPI chip-select
14	`cs_0`	Output	MFRC522 SPI chip-select
15	`spi_miso`	Input	SPI data from peripheral
16	`spi_mosi`	Output	SPI data to peripheral
17	`spi_sclk`	Output	SPI clock
20	`status_unlock`	Output	Green LED / relay unlock
21	`status_fault`	Output	Red LED (authentication error)
22	`status_busy`	Output	Yellow LED (transaction in progress)

`chip_core.sv` Routes signals between pad cells and the main logic core. Holds the pin-to-signal mapping table as comments for traceability.

`main_core.v` Integration level: instantiates the SPI master, the NFC interface, the EEPROM interface, and the LAYR core. Multiplexes the SPI master between the two consumers: the EEPROM interface has priority during its power-on read sequence; after that, the NFC interface takes control. The NFC interface is kept in reset (`rst | eeprom_busy`) while the EEPROM is loading, ensuring that NFC activity cannot interfere with key retrieval.

4 Security Analysis and Protocol Design

4.1 Threat Model

We consider an adversary with the following capabilities:

- *Passive eavesdropping.* The attacker can observe all wireless messages between card and reader.
- *Active relay.* The attacker can inject, modify, or replay recorded messages within the same session or across sessions (replay attack).

We explicitly *exclude* the following threats from the scope of this level: Side-channel attacks on the AES core (Level 2 requirement), fault-injection attacks (Level 3 requirement), and compromise of the EEPROM key storage (explicitly excluded by the challenge specification).

4.2 LAYR Level 1 Protocol

The LAYR Authenticated Identification Protocol [18] is a symmetric mutual challenge-response scheme executed over ISO 14443-4 APDU commands. Both parties share a 128-bit pre-shared key k_{psk}.

Protocol flow. The chip (terminal) initiates the exchange after detecting a card:

(1) **SELECT AID.** The terminal selects the LAYR applet by its application identifier `F000000CDC01`.
(2) **AUTH_INIT (INS=0x10).** The card draws a random 8-byte nonce r_C, computes $c_C = \text{AES}_{k_{\text{psk}}}(r_C \parallel 0^{64})$, and returns c_C. The terminal decrypts c_C and recovers r_C. This proves (to the terminal) that the card knows k_{psk}.
(3) **AUTH (INS=0x11).** The terminal generates its own 8-byte nonce r_T using a counter-based CTR construction ($r_T = \text{AES}_{k_{\text{psk}}}(0^{64} \parallel ctr)_{\text{lo}64}$), increments the counter, and computes $c_T = \text{AES}_{k_{\text{psk}}}(r_T \parallel r_C)$. The ephemeral session key is formed as $k_{\text{eph}} = r_C \parallel r_T$. The terminal sends c_T to the card.

(4) The card decrypts c_T to recover r_T and r_C. It verifies that the received r_C matches the value it generated in step 2, proving that the terminal knows k_{psk}. The card also forms $k_{\text{eph}} = r_C \parallel r_T$. It responds with

$$\text{AES}_{k_{\text{eph}}}(\texttt{"AUTH_SUCCESS"} \| 0^{32}).$$

(5) **GET_ID (INS=0x12).** The terminal sends a GET_ID command. The card responds with $\text{AES}_{k_{\text{eph}}}(id)$, where id is the 128-bit card identity. The status word `0x9001` instead of `0x9000` indicates that the card has a new key ready for rollover.

(6) The terminal decrypts the identity and checks it against the six-slot whitelist loaded from the EEPROM. If the identity is in the list, the `status_unlock` output is asserted for 3 seconds.

Security properties. Freshness of r_C is guaranteed by the card's random number generator. Freshness of r_T is guaranteed by the monotonic counter stored in non-volatile EEPROM memory: even after a power cycle the counter never repeats, so the terminal nonce is never reused. The ephemeral key k_{eph} is session-unique; intercepting one session does not help an attacker in a future session. The AUTH step in direction terminal → card also serves as a proof of key knowledge, providing mutual authentication.

Resistance to replay. Each session uses a fresh r_C (random) and a fresh r_T (counter-derived). Replaying a captured c_T from a previous session will fail at the card's r_C verification because r_C will differ. Replaying a captured session entirely is prevented because r_C is a fresh random value chosen by the card.

4.3 Key Rollover

To support periodic key rotation without manual EEPROM reprogramming, the protocol includes an optional key-rollover extension. When the card returns `SW=0x9001` in step 5, the terminal sends an additional **GET_NEW_KEY (INS=0x21)** command. The card responds with $\text{AES}_{k_{\text{eph}}}(k_{\text{new}})$, the new key encrypted under the established ephemeral key. The terminal decrypts k_{new}, writes it to the EEPROM (replacing k_{psk}), and only then asserts the unlock signal.

This mechanism ensures that a key update only takes effect after a successfully authenticated transaction, preventing denial-of-service through rogue key injection (which would require knowing k_{psk} to forge k_{eph}).

5 RTL Implementation

5.1 Shared SPI Master (`spi_master.v`)

A single SPI master instance is shared between the MFRC522 and the AT25010B by a simple priority multiplexer in `main_core.v`: the EEPROM interface holds priority during its power-on read; afterwards the NFC interface drives the bus. The master operates in SPI Mode 0 (CPOL=0, CPHA=0), transmits MSB first, and exposes a one-byte handshake interface (`start_xfer`, `tx_byte`, `xfer_done`, `rx_byte`). The SPI clock is derived from the 32 MHz system clock by a counter-based divider, configurable at synthesis time via the `CLK_HZ` and `SPI_HZ` parameters. The default configuration produces a 4 MHz SPI clock (8-cycle period).

5.2 NFC Front-End Interface (`mfrc522_apdu_interface.v`)

This module is responsible for all communication with the MFRC522 NFC reader chip and the underlying card. It operates in four main phases, shown in fig. 7:

Phase 0 – Reset and initialization. To get the reader into a state where it can detect cards, the module issues a `SoftReset` to the MFRC522, writes the operating registers (modulation, antenna, gain, timer), and reads back the `Version` register to confirm the chip is present before advancing to card detection.

Phase 1 – ISO 14443-A card detection. The module periodically sends a REQA command (`0x26`) through the MFRC522's Transceive function. On receiving an ATQA response it executes the anti-collision loop (single-cascade UID) and the SELECT command to bring the card into ACTIVE state.

Phase 2 – ISO 14443-4 link bring-up. After a successful SELECT, the module sends a RATS (Request for Answer To Select) frame with parameters suitable for a JavaCard (FSDI=8, DID=0). The module receives the ATS (Answer To Select), enables ISO 14443-4 CRC mode on both transmit and receive paths of the MFRC522, re-configures the MFRC522 timer registers (`TMode` and `TPrescaler`) explicitly for the APDU phase, and then waits for the mandatory Start-up Frame Guard Time (SFGT) of approximately 10 ms before asserting `link_ready`. The SFGT wait states (`APDU_CFG2_TMODE`, `APDU_CFG2_TPRESCALER`, `APDU_ATS_SFGT_WAIT`) are explicitly excluded from the `link_ready` condition so that the LAYR core cannot begin an APDU transaction before the card is ready to receive the first frame.

Phase 3 - APDU exchange. Once `link_ready` is asserted, the module transparently forwards APDU bytes between the application layer (`app_tx`/ `app_rx`) and the card, framing them in ISO 14443-4 I-blocks. It handles block numbering, WTX (waiting-time-extension) acknowledge frames, and error detection via the MFRC522's error register. WTX frames require a minimum of two bytes (PCB + WTXM), while normal I-blocks require at least three (PCB + SW1 + SW2); the receive length check was corrected accordingly to avoid rejecting valid WTX frames. When a timer interrupt fires without a simultaneously pending error bit (TimerIRq alone), the module performs a soft restart of the card-detection loop rather than signalling a hard fault, which enables graceful recovery when the card does not respond to RATS or to an individual APDU. The `block_num` counter is explicitly reset to zero on every soft or hard restart to avoid I-block sequence-number mismatch on the next transaction. The module asserts `hard_fault` and restarts from the hard-reset state only on unrecoverable (CRC or framing) errors.

5.3 AES Engine (`aes_core.v` / `aes_iterative.v`)

The AES engine is a compact iterative design that executes one AES-128 round per clock cycle. It supports both encryption and decryption, selected by a `mode` signal. The key schedule is also computed iteratively (one round key per cycle) to avoid a long combinatorial path.

S-box implementation. To minimise area, the S-box uses the composite-field construction whose mathematical theory was developed by Canright [2]. The concrete Verilog implementation is taken from the open-source repository published by GitHub user *coorus* [3]. This S-box operates over GF(2^4) / GF(2^2) tower-field arithmetic and avoids all ROM-based lookup tables, resulting in significantly fewer standard cells than a conventional LUT approach.

Timing. A complete AES-128 encryption or decryption takes 11 clock cycles (initial key addition + 10 rounds). At 32 MHz this corresponds to approximately 344 ns per operation.

Design history. The first AES integration used a publicly available single-cycle Verilog implementation [5] that computes a full AES-128 block in one clock cycle via a completely unrolled combinatorial datapath. Although functionally correct, this design proved far too large for the available area budget when instantiated twice (once per authentication

direction). It was replaced by a custom iterative implementation (`aes_iterative.v`) that processes one round per cycle and uses the *coorus* S-box for compactness.

A further optimisation removed the need for two AES instances: the terminal nonce r_T is derived via the counter-based CTR construction ($r_T = \mathrm{AES}_{k_{\mathrm{psk}}}(0^{64} \| ctr)_{\mathrm{lo64}}$), which reuses the single AES core that is already needed for the authentication steps. This also eliminated a separate LFSR-based nonce generator that had been present in an intermediate design.

5.4 EEPROM Interface (`at25010_interface.v`)

The AT25010B is a 128-byte (128 × 8-bit) SPI EEPROM [15]. Table 2 shows the memory layout used by the chip.

Table 2. EEPROM memory layout (AT25010B, 128 bytes total).

Address (hex)	Size (bytes)	Width	Content
`0x00–0x0F`	16	128 bit	AES pre-shared key
`0x10–0x17`	8	64 bit	Monotonic counter (nonce base)
`0x18–0x77`	96	6×128 bit	Whitelist (ID slot 0…5)
`0x78–0x7F`	8	–	Reserved

On power-up (or after every reset), the EEPROM interface performs a single sequential read of all 120 bytes starting from address `0x00`. The LAYR core and NFC interface are held in reset until this read is complete and all three valid signals (`key_valid`, `counter_valid`, `ids_valid`) are asserted.

Writes are carried out in 8-byte pages, as mandated by the AT25010B page-write architecture. Each page write is preceded by a WREN (Write Enable) command followed by a status register read to verify that the Write Enable Latch (WEL) bit is set; if it is not, the write is abandoned and a fault flag is raised. Write completion is determined by polling the Write-In-Progress (WIP) bit of the status register.

The EEPROM state machine supports three write paths:

- *Key write* (2 pages, addresses `0x00–0x0F`): triggered by the key-rollover mechanism.
- *Counter write* (1 page, addresses `0x10–0x17`): triggered after every successful or failed authentication transaction to persist the incremented counter.
- *ID-list write* (12 pages, addresses `0x18–0x77`): provided for future use; not currently triggered by any internal module.

Figure 2 illustrates the EEPROM state machine.

5.5 LAYR Authentication Core (`layr_core.v`)

The LAYR core implements the complete protocol state machine described in Section 4.2. Figure 3 shows all 29 states grouped by function.

Status outputs. Three status output signals are driven from the LAYR core and the EEPROM interface:

- `status_busy`: asserted whenever the LAYR core or EEPROM interface is active.
- `status_unlock`: asserted for 3 000 ms after a successful authentication (and ID whitelist check).
- `status_fault`: asserted for 3 000 ms on any authentication failure (wrong key, ID not in whitelist, protocol error, or NFC link fault).

Counter persistence. After every authentication attempt (success or failure) the incremented nonce counter is written back to the EEPROM. This operation is deliberately deferred until after the HOLD state to minimise the latency perceived by the user at the door. The LAYR core tracks whether the counter has been modified (`counter_dirty`) and triggers the write-back via `counter_write_req` before returning to IDLE.

ID whitelist. The 96-byte whitelist region accommodates up to six 128-bit card identities (ID 0 through ID 5). A slot is considered empty if all 128 bits are zero. During development, six default identities were pre-programmed into the EEPROM for testing purposes.

Link recovery. If the NFC link drops unexpectedly (the card is removed mid-transaction), the LAYR core detects the falling edge of `link_ready` and immediately returns to IDLE—provided it is not in a critical section that must not be interrupted (such as an EEPROM write). This prevents the machine from blocking indefinitely on missing APDU responses.

6 Android Simulator Application

No physical JavaCard was available during the majority of development; one was only obtained at the very end of the project. To enable software-level testing throughout the development process, a companion Android application was developed that emulates the card side of the LAYR protocol using Android's Host Card Emulation (HCE) framework [19].

The application registers the AID `F000000CDC01` with the Android NFC subsystem and implements the complete LAYR Level 1 card protocol in a `HostApduService` (`CardService.kt`). Table 3 summarises the implemented APDU commands.

Table 3. APDU commands implemented by the Android simulator (class `CardService`).

Command	CLA	INS	Card behaviour
SELECT AID	0x00	0xA4	Reset state; return 0x9000
AUTH_INIT	0x80	0x10	Generate random r_C; return $\mathrm{AES}_{k_{\mathrm{psk}}}(r_C \| 0^{64})$
AUTH	0x80	0x11	Verify r_C; compute k_{eph}; return $\mathrm{AES}_{k_{\mathrm{eph}}}(\texttt{AUTH_SUCCESS})$
GET_ID	0x80	0x12	Return $\mathrm{AES}_{k_{\mathrm{eph}}}(id)$; SW=0x9001 if new key pending
GET_NEW_KEY	0x80	0x21	Return $\mathrm{AES}_{k_{\mathrm{eph}}}(k_{\mathrm{new}})$

The user interface consists of three fragments (fig. 4):

- *Key*: the main fragment for regular Level 1 authentication; displays the current transaction status (idle, communicating, success, failure) and the active PSK.
- *Rollover*: configures the new key to be distributed in the next successful transaction and enables rollover mode, causing GET_ID to return `SW=0x9001` so the terminal requests and stores the new key.
- *Key Management*: allows updating the card's PSK by entering a 32-character hexadecimal string.

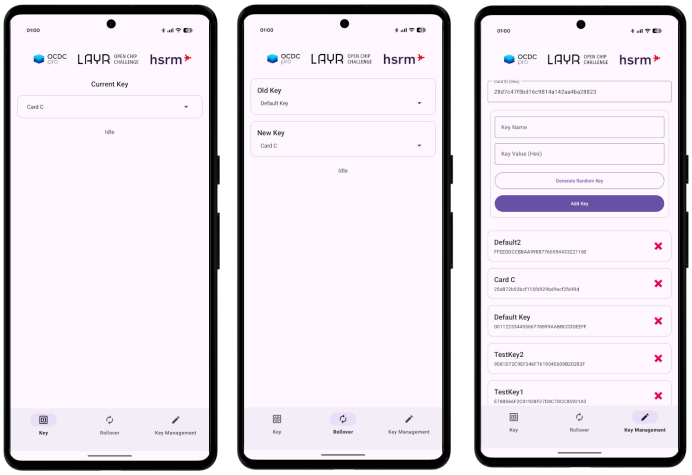

Fig. 4. Android LAYR Simulator application showing the three fragments: Key (authentication status and active PSK), Rollover (key distribution) and Key Management (PSK update).

The application is intentionally kept simple—all cryptographic operations use the standard Android `javax.crypto` AES/ECB cipher—to match the hardware behaviour exactly and serve as a ground truth for debugging.

Simulator limitation: JavaCard compatibility. A third-party LAYR protocol implementation rejected the Android HCE application entirely. The likely cause is that the implementation targeted JavaCards exclusively and relied on card-specific behaviour that Android HCE does not reproduce exactly – for example, differences in APDU framing, ATS parameters, or response timing. The Guardian Chip design was verified against both a physical JavaCard and the Android HCE simulator without modification.

7 Physical Design

7.1 Technology

The chip targets the IHP SG13G2 130 nm BiCMOS open-source process [9]. SG13G2 provides a 130 nm CMOS baseline layer stack suitable for digital logic and has been released with a fully open-source PDK under the Apache 2.0 license. The submitted GDS will be physically manufactured by IHP, with fabrication expected in 2026 [10].

7.2 Pad Ring

The `chip_top.sv` module instantiates the following IHP pad cell types:

- `sg13g2_IOPadIn` for `sys_clk`, `rst`, `uart_clk` (unused), `uart_rx` (unused), and `spi_miso`.
- `sg13g2_IOPadOut30mA` for `uart_tx` (idle-high), `spi_sclk`, `spi_mosi`, `cs_0`, `cs_1`, and the three status outputs.
- `sg13g2_IOPadInOut30mA` for the five user-IO bidirectional pads (currently held as inputs).
- Power pads: two `IOPadVdd`/`IOPadVss` pairs and one `IOPadIOVdd`/`IOPadIOVss` pair.

7.3 Test Setup

All hardware tests were performed on an ECP5-based ULX3S FPGA board as an interim development platform. A custom 3D-printed case (FreeCAD source in `CAD/`) was produced to mechanically integrate the ULX3S board, the MFRC522 reader, the EEPROM breakout, the relay and the status LEDs into a demonstration unit (Figure 5).

Fig. 5. 3D-printed demonstration enclosure housing the ULX3S FPGA development board, MFRC522 NFC reader, AT25010B EEPROM, relay, and status LEDs. CAD files (FreeCAD + STL) are located in `CAD/`.

8 Development Process

This section describes the development timeline, the main challenges encountered, and how they were resolved. The project was carried out by a single person after the originally planned team members withdrew from the challenge early in the project. While this reduced the available resources, it also gave full architectural control over all design decisions from the start. It should be noted that the author had no prior experience with ASIC design, Verilog/SystemVerilog, or SPI peripherals at the start of the project. The learning curve across all three domains simultaneously was steep and accounts for several of the detours and delays described in this section.

8.1 Phase 0: Simulation-First Approach and Its Limits

The project began in November 2025 with the intention to follow a simulation-driven development methodology. A set of testbenches was written using the Icarus Verilog simulator and the cocotb framework. The idea was to verify each hardware interface module in simulation before touching real hardware.

In practice, this approach did not work for the hardware interface modules. Correctly modelling the timing behaviour of the MFRC522 NFC reader and the AT25010B EEPROM in a software simulation requires detailed, cycle-accurate behavioural models of those devices—models that either do not exist as open-source resources or would require more

effort to build than the hardware itself. As a consequence, *none* of the simulation-based testbench code from this phase made it into the final design. The lesson drawn was that for hardware interface modules, simulation is a useful supplement but cannot replace testing on the actual target hardware.

8.2 Phase 1: Infrastructure and Basic SPI Communication

Because no official LAYR hardware kit was received, all external components (MFRC522 reader, AT25010B EEPROM breakout, relay, door lock and LEDs) had to be procured and assembled independently. A custom 3D-printed enclosure was designed in FreeCAD and printed to integrate all components into a single demonstration unit (see Section 7.3 and fig. 5).

The initial development platform was an ECP5-based ULX3S FPGA board. By late January 2026, basic SPI communication with the MFRC522 was established, and the chip was successfully reading the MFRC522 firmware version register as a first integration test. Basic ISO 14443-A card detection with the anticollision sequence was working shortly after.

First milestone: LAYR Level 0. With card detection working, a minimal access-control loop was implemented that read the card's ISO 14443-A UID directly from the MFRC522 and compared it against a hard-coded identifier. Strictly speaking, LAYR Level 0 requires APDU-based communication over ISO 14443-4; the UID check implemented here does not satisfy this requirement, as the UID is obtained during the anticollision procedure without establishing a transport layer. This nonetheless constituted a first functional access-control demonstrator by the end of January 2026 and confirmed that the reader hardware and SPI driver were fundamentally sound. Implementing full APDU-based mutual authentication—the LAYR Level 1 requirement—proved to be a far harder problem, as the following phase describes.

Debugging without instruments. During this phase the only available debugging tools were the eight general-purpose LEDs on the ULX3S FPGA board (in addition to the three dedicated status LEDs for unlock, busy, and fault). Protocol state was encoded into bit patterns on these LEDs to indicate which state the NFC detector state machine had reached. This method proved sufficient for coarse-grained progress tracking but made it nearly impossible to diagnose timing issues or inspect individual SPI bytes.

8.3 Phase 2: The APDU Dead-End and the Bootstrapping Problem

The central challenge of the project was implementing the ISO 14443-4 transport layer (APDU exchange) over the MFRC522. This required two pieces of software to work correctly at the same time: the MFRC522 driver on the FPGA, and the Android HCE application on the card side.

The bootstrapping problem. Both components were developed simultaneously without an independent reference implementation to verify either one against. The MFRC522 driver could not be tested without a working card emulator, and the Android HCE application could not be tested without a working NFC terminal. This mutual dependency created a bootstrapping problem: any failure was ambiguous because the error could be in either component.

The Android application was written under the assumption that it was correct, and the MFRC522 driver was then developed against that assumption. This led to an extended series of failed iterations without clear diagnostic information—the FPGA would signal a fault LED and provide no further detail.

Attempted detour: Verilog PN532 interface. With the Android HCE application producing no observable output and no independent reference available, a different approach was explored: replacing the MFRC522 with a PN532 NFC controller. The PN532 implements the ISO 14443-4 transport layer in firmware, exposing a higher-level host interface (HSU/SPI/I^2C) that abstracts away raw frame handling. The hope was that this would significantly reduce the protocol complexity on the FPGA side—the host would only need to issue high-level commands and receive complete APDUs, rather than managing frame-level details manually.

A dedicated Verilog interface module for the PN532 was written and initially brought up. However, this approach was ultimately abandoned: integrating yet another unproven component into an already failing system risked compounding the ambiguity, and the time investment of adapting the surrounding RTL was difficult to justify while the root cause of the original failures remained unknown.

Resolution: PN532 as reference reader. The deadlock was eventually broken by using the PN532 in a different role: not as a replacement for the MFRC522 in the FPGA design, but as an independent reference terminal. An Arduino sketch using the publicly available Seeed PN532 library (`seeed_pn532`) [21] was written to send APDU commands to the Android phone via ISO 14443-4. This provided, for the first time, an independent terminal that could exercise the Android HCE application in isolation.

The test immediately revealed that the Android application had a configuration error: the `HostApduService` was not correctly registered in the Android manifest, so the operating system never routed NFC APDUs to the application. Once this was corrected, the PN532 sketch confirmed that the APDU protocol logic in the application was functionally correct.

At approximately the same time, a logic analyser became available. With both problems resolved—a confirmed working card emulator and byte-level SPI visibility—the MFRC522 APDU driver was debugged and brought to a working state within a short period. The first end-to-end APDU exchange was achieved at the end of February 2026.

Lesson learned. The bootstrapping problem and the subsequent detour through a Verilog PN532 interface together cost several weeks of development time. In hindsight, procuring an independent reference NFC terminal at the very start of the project—before any custom driver code was written— would have allowed each component to be verified in isolation and would likely have compressed this phase to a matter of days. For future projects of this kind, establishing an independent ground-truth implementation of every external interface before beginning the primary development is strongly recommended.

8.4 Phase 3: LAYR Protocol and System Integration

With a working ISO 14443-4 transport layer, the LAYR authentication protocol was implemented in the `layr_core` module. A first version was tested with a hard-coded dummy key and confirmed working in early March 2026.

The subsequent work focused on integration and area optimisation:

- The AT25010B EEPROM interface was integrated so that the key, counter, and ID whitelist are loaded from non-volatile storage on power-up.
- A key-rollover mechanism was added on top of the standard LAYR Level 1 protocol, allowing in-band key updates as part of an authenticated transaction.
- The single-cycle AES implementation was replaced by the custom iterative engine described in Section 5.3, and the LFSR-based nonce generator was removed in favour of the AES-CTR construction.

- The *coorus* composite-field S-box was integrated to further reduce the area of the AES core.
- An ID whitelist check with three pre-programmed default identities was added to the LAYR core.

8.5 Phase 3b: JavaCard Compatibility and EEPROM Initialisation

Once the Android HCE application worked reliably as a card emulator, a physical JavaCard running the reference LAYR applet was obtained for final validation. Testing with the JavaCard exposed several timing and protocol issues in the `mfrc522_apdu_interface` that had gone undetected with the Android emulator, because Android's HCE framework is more tolerant of non-conforming ISO 14443-4 behaviour than a hardware card. Four specific fixes were applied to the module. (1) The I-block sequence number was not reset on card-detection restart, causing the JavaCard to reject the first frame with a protocol error; `block_num` is now zeroed on every restart. (2) The ISO 14443-4 Start-up Frame Guard Time (SFGT) was not observed: the JavaCard required a minimum 4.8 ms gap after the ATS response, while the Android emulator accepted frames immediately. A 10 ms SFGT wait state was inserted, and the MFRC522 timer registers were reconfigured explicitly after the ATS exchange. (3) A card moved briefly out of range fires only `TimerIRq` without an error bit; the original code treated this as a hard fault, triggering a 3 second lockout. The fix performs a soft card-detection restart on timeout-only events. (4) The receive-length check required at least three bytes, rejecting valid two-byte S(WTX) frames; the minimum was reduced to two bytes.

After these four fixes, the complete protocol ran successfully on both the Android HCE emulator and the physical JavaCard.

EEPROM initialisation. To pre-programme the AT25010B EEPROM with the default key, nonce counter, and card-identity whitelist, a dedicated FPGA bitstream was built from the test module `fpga/test/at25010_write_test_top.v`. This module burns a fixed set of parameters into the EEPROM at power-up: the AES pre-shared key (`DEBUG_KEY`), an all-zero nonce counter, and six whitelist entries. The first entry (`ID_1`) is a fixed test identity (`0x00. . . 01`). The remaining five entries (`ID_2` through `ID_6`) contain the LAYR protocol identities of the cards and devices used during hardware validation. These identities are LAYR-layer values transmitted encrypted within the APDU exchange and are distinct from the ISO 14443-A UIDs exchanged during anticollision. The bitstream was programmed once onto the ULX3S board with the EEPROM connected; subsequent FPGA reconfigurations with the production bitstream then load the pre-written values at startup.

8.6 Phase 4: Tape-Out Preparation

Tape-out preparation involved adapting the RTL to the IHP SG13G2 pad cell library and iteratively resolving physical verification failures that only manifest in the full back-end flow. The two main challenges encountered—metal density fill and antenna violations—are described in the following subsections.

8.6.1 Metal Density Fill. The IHP SG13G2 PDK mandates a minimum metal fill density of 25 % and a maximum of 75 % in every 800×800 µm window (rules `M2Fil.h`/`M3Fil.h` and `M2Fil.k`/`M3Fil.k`) [9]. After place-and-route, the LibreLane KLayout filler step [11] placed 1.0×1.0 µm fill shapes on Metal 2 and Metal 3 with a default fill-to-fill spacing of 1.5 µm (M2) and 2.0 µm (M3). Despite this, 13 local density violations remained on Metal 2 and Metal 3 in the high-density routing region (400–2000 µm from the core origin); no violations occurred on any other metal layer. The default KLayout filler is unable to correctly fill those tiles. As an intermediate measure, reducing the fill-to-fill spacing in the PDK filler macro (`sg13g2_filler_Metal.lym`) from 1.5 µm to the DRC-minimum of 0.42 µm increased coverage and reduced the

violation count, but did not fully eliminate them. Ultimately, these PDK-level changes proved unnecessary once the gdsfill-based approach described below was adopted, and the macro was left at its original default parameters.

To close the remaining violations, the open-source gdsfill [1] tool was evaluated as a secondary fill pass. gdsfill applies a combination of Track and Square tiling algorithms on configurable 400 µm tiles to reach a user-specified target density, using fill shapes of at least 1.0 µm width to satisfy the MFil.a1 minimum fill width rule. Running it directly on the LibreLane output, however, introduced over 200 000 new M2Fil.a2/M3Fil.a2 errors due to a double-fill interaction: gdsfill placed shapes on top of existing KLayout fill, and the DRC rule fires when merged shapes exceed 5 µm.

The fix consisted of three steps. First, a dedicated KLayout script (erase_m2m3_fill.py) was written to selectively erase the Metal 2 and Metal 3 filler layers from the step-63 GDS before invoking gdsfill, leaving the M1, M4, M5, TM1, and TM2 fill produced by the KLayout pass intact. Second, the tile-boundary keep-out expansion in gdsfill's prepare.py was increased from the nominal DRC spacing of 0.42 µm to 0.50 µm to account for routing metal that straddles tile boundaries and is therefore invisible to the fill-placement algorithm of the adjacent tile. Third, the target fill density in gdsfill_config.yaml was raised from 35 % to 45 % so that tiles whose routing density already exceeded 35 % would not be skipped, preventing M2Fil.h violations that would arise if gdsfill left them unfilled. The complete procedure was integrated into the build system as the gdsfill-clean-resume Makefile target, which backs up the filler GDS, erases M2/M3 fill, runs gdsfill, and then resumes the LibreLane flow from the density-check step onward. To run the entire flow from scratch in a single step, the librelane-full-gdsfill-clean target combines the full synthesis and place-and-route run with the subsequent gdsfill pass. Notably, no modifications to the PDK filler macro were required; the default KLayout filler parameters remain unchanged and handle all other metal layers correctly. The resulting chip layout is shown in Figure 6.

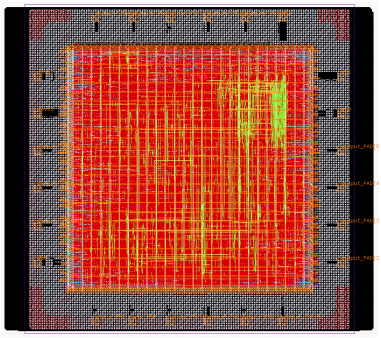

Fig. 6. Final GDS of the Guardian Chip as viewed in KLayout after completion of the full LibreLane flow including metal density fill. The pad ring with 24 I/O pads is visible around the perimeter; the core logic occupies the central area.

8.6.2 Floating Nets and Antenna Violations. OpenROAD's post-routing antenna check reported 8 pin and 8 net violations. Inspection of the OpenROAD metrics revealed timing__drv__floating__nets: 4, indicating that the router could not repair antenna effects on nets without a fanout. The root cause was in chip_core.sv: the two unused input pads uart_clk and uart_rx were connected only to an AND-reduction expression assigned to a local signal _unused. Yosys optimises such dead-end nets away during synthesis, leaving the physical input pad nets without any downstream load. The fix was to route each unused input directly to a corresponding spare output pad—uart_clk to user_io_0 and uart_rx to uart_tx—ensuring that the nets connect two physical I/O cells and cannot be eliminated. After this change, the KLayout antenna check reported zero violations (klayout__antenna_error__count: 0).

9 Verification

Functional verification was performed at two levels: RTL simulation using the cocotb Python-based testbench framework, and hardware validation on the ULX3S FPGA board with the Android simulator application.

RTL simulation. The testbench directory (`cocotb/`) contains two Python-based cocotb testbenches:

- `test_aes_core.py` – unit tests for the AES engine using official NIST FIPS 197 vectors [16] plus round-trip encrypt/decrypt checks.
- `test_layr_core.py` – integration tests for the LAYR authentication core, covering successful authentication, rejected card (ID not in whitelist), wrong PSK, key rollover, and link-drop mid-transaction recovery. The testbench acts as both the NFC link and a software implementation of the card side using Python AES.

Hardware validation. The Android HCE application was used as a software card against the FPGA prototype. Because the application logs every received and transmitted APDU, it was possible to verify the complete protocol flow byte-by-byte. Test cases covered:

(1) Successful authentication with a whitelisted identity.
(2) Rejected card (identity not in whitelist).
(3) Wrong PSK key (AUTH step fails).
(4) Authentication followed by key rollover.
(5) Card removal mid-transaction (link-drop recovery).

10 Discussion

Security level. The current design fulfils LAYR Level 1 requirements. Higher levels require additional hardware countermeasures against physical attacks (e.g. power analysis or fault injection) that were out of scope for this project.

Relay attacks. The LAYR Level 1 threat model explicitly includes active relay attacks [8], in which an adversary transparently forwards NFC frames between a legitimate reader and a distant card. The mutual challenge-response scheme prevents a *replay* of a previously recorded session, but does not inherently bound the physical distance between card and reader during a live session. A practical relay is constrained by the added round-trip latency introduced by the relay channel; tightly timing the challenge-response exchange at the application layer could provide a degree of distance bounding. This is not currently implemented and would be a meaningful extension for deployments where the relay threat is significant.

Area and performance. The iterative AES design processes one round per clock cycle, resulting in an 11-cycle latency at 32 MHz (approximately 344 ns). A fully unrolled pipeline would reduce latency at the cost of area; the current balance was chosen to fit within the QFN-24 area budget and on the FPGA. Post-layout metrics obtained from the completed LibreLane back-end run are summarised in table 4. The design comprises 49 208 standard cells (of which 4 833 are sequential) plus 10 275 timing-repair buffers inserted by OpenROAD. The 2.699×2.699 mm die houses a 1.943×1.943 mm core region at a standard-cell utilisation of 18.6 %, leaving substantial routing headroom. The chip is designed to operate at 32 MHz, which is the clock frequency assumed by all internal timing parameters (SPI baud rate, millisecond timers, unlock duration). The LibreLane synthesis constraint was deliberately set to 50 MHz to drive OpenROAD towards tighter timing closure and provide margin. At the slow corner (1.08 V, 125 °C) setup timing is not met at 50 MHz; at

32 MHz the available margin is approximately 13.7 ns, which is sufficient for operation in that corner. Estimated total power consumption at nominal conditions is 10.3 mW.

Table 4. Post-layout implementation summary (IHP SG13G2, LibreLane flow).

Metric	Value
Die size	2.699×2.699 mm (7.28 mm^2)
Core area	1.943×1.943 mm (3.77 mm^2)
Standard-cell count	49 208
of which sequential	4 833
Standard-cell utilisation	18.6 %
Operating frequency	32 MHz
Synthesis timing target	50 MHz (20 ns period)
Critical path (typ. corner)	17.5 ns
Setup slack @ 50 MHz (typ.)	+2.5 ns
Max. frequency (typ. corner)	57 MHz
Total wirelength	4 401 mm
Estimated power (nom. typ.)	10.3 mW

Nonce counter overflow. The 64-bit monotonic counter supports 2^{64} authentication transactions before wrapping around. Even if an adversary deliberately triggers failed authentication attempts to force the counter forward, the enforced minimum inter-transaction delay of 3 seconds means that reaching a wrap-around would take an astronomically long time, rendering this attack entirely impractical.

11 Conclusion

This report presented the Guardian Chip, a compact NFC access-control ASIC designed for the LAYR Open Chip Challenge 25/26. The chip implements the LAYR Level 1 Authenticated Identification Protocol over an ISO 14443-4 link, providing mutual AES-128 challenge-response authentication and session-key-protected ID transfer.

Key design choices include a shared SPI bus with EEPROM-first priority scheduling, a single iterative AES-128 core using the *coorus* composite-field S-box (based on Canright's mathematics) for area efficiency, a counter-based terminal nonce to avoid a dedicated random number generator, and a key-rollover mechanism designed as an additional feature on top of the LAYR Level 1 specification that binds key updates to successful authentication transactions.

A companion Android HCE application provides a fully software-based card emulator for development and testing without requiring physical JavaCards. The design targets tape-out in the IHP SG13G2 130 nm process using the LibreLane open-source ASIC flow.

Acknowledgements

The author thanks Prof. Dr. Steffen Reith (Hochschule RheinMain) for supervision and guidance throughout this project.

AI Usage Disclosure

Parts of this work made use of large language model (LLM) assistance. Specifically, an LLM was used to generate initial Verilog/SystemVerilog code that served as a starting point for subsequent manual debugging, bug fixes, and improvements; to generate initial Android application code (HCE service, APDU handling) that similarly served as a

basis for manual refinement; to assist with interpreting SPI signal traces and cross-referencing them against peripheral datasheets; and to draft and refine portions of this report. All generated code was reviewed, tested, and modified by the author.

References

[1] aesc-silicon. 2026. `gdsfill`: Metal Density Fill Tool for Open-Source PDKs. GitHub: https://github.com/aesc-silicon/gdsfill. Supports IHP SG13G2 and Sky130; Track and Square tiling algorithms.

[2] David Canright. 2005. A Very Compact Rijndael S-box. In *Cryptographic Hardware and Embedded Systems – CHES 2005 (Lecture Notes in Computer Science, Vol. 3659)*. Springer, 441–455. doi:10.1007/11545262_32

[3] coorus. 2015. Canright AES S-boxes – Verilog Implementation. GitHub: https://github.com/coruus/canright-aes-sboxes. Verilog implementation of Canright's compact Rijndael S-box.

[4] Danny Dolev and Andrew C. Yao. 1983. On the Security of Public Key Protocols. In *IEEE Transactions on Information Theory*, Vol. 29. 198–208. doi:10.1109/TIT.1983.1056650

[5] Michael Ehab. 2022. AES-Verilog: AES-128 Hardware Implementation in Verilog. GitHub: https://github.com/michaelehab/AES-Verilog. Single-cycle, fully unrolled combinatorial AES-128 datapath.

[6] EMVCo. 2014. EMV Contactless Specifications for Payment Systems – Book C-2: Kernel 2 Specification. *EMVCo Specification* (2014). Version 2.5.

[7] Martin Feldhofer, Sandra Dominikus, and Johannes Wolkerstorfer. 2004. Strong Authentication for RFID Systems Using the AES Algorithm. *Lecture Notes in Computer Science* 3156 (2004), 357–370. doi:10.1007/978-3-540-28632-5_26 Cryptographic Hardware and Embedded Systems – CHES 2004.

[8] Gerhard P. Hancke and Markus G. Kuhn. 2005. An RFID Distance Bounding Protocol. In *First International Conference on Security and Privacy for Emerging Areas in Communications Networks (SecureComm)*. IEEE, 67–73. doi:10.1109/SECURECOMM.2005.56

[9] IHP – Leibniz-Institut für innovative Mikroelektronik. 2023. IHP Open Source PDK: SG13G2 130 nm BiCMOS Technology. Open-Source Repository. GitHub: https://github.com/IHP-GmbH/IHP-Open-PDK.

[10] IHP – Leibniz-Institut für innovative Mikroelektronik. 2025. IHP Microelectronics. https://www.ihp-microelectronics.com/.

[11] IHP-GmbH. 2024. LibreLane Full-Chip Flow Template for IHP SG13G2 OpenPDK. GitHub: https://github.com/IHP-GmbH/ihp-sg13g2-librelane-template. Apache-2.0 license.

[12] ISO/IEC. 2018. *ISO/IEC 14443-3:2018 – Identification cards – Contactless integrated circuit cards – Proximity cards – Part 3: Initialization and anticollision*. International Standard 14443-3. International Organization for Standardization.

[13] ISO/IEC. 2018. *ISO/IEC 14443-4:2018 – Identification cards – Contactless integrated circuit cards – Proximity cards – Part 4: Transmission protocol*. International Standard 14443-4. International Organization for Standardization.

[14] LAYR Team / OCDC. 2025. LAYR Open Chip Challenge 25/26. GitHub: https://github.com/OCDCpro/LAYR. Challenge description and hardware kit specification.

[15] Microchip Technology. 2013. *AT25010B/020B/040B: 1K/2K/4K SPI Bus Serial EEPROM*. Microchip Technology Inc. https://ww1.microchip.com/downloads/en/devicedoc/atmel-8707-seeprom-at25010b-020b-040b-datasheet.pdf.

[16] National Institute of Standards and Technology. 2001. *FIPS PUB 197: Advanced Encryption Standard (AES)*. Federal Information Processing Standard 197. National Institute of Standards and Technology, Gaithersburg, MD, USA. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf.

[17] NXP Semiconductors. 2016. *MFRC522: Standard performance MIFARE and NTAG Frontend* (rev. 3.9 ed.). NXP Semiconductors. https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf.

[18] OCDC. 2025. LAYR JavaCard Applet. GitHub: https://github.com/OCDCpro/javacard-applet. Reference implementation of the LAYR protocol on JavaCard.

[19] Michael Roland, Josef Langer, and Josef Scharinger. 2013. Applying Relay Attacks to Google Wallet. In *5th International Workshop on Near Field Communication (NFC)*. IEEE. doi:10.1109/NFC.2013.6482441

[20] Akashi Satoh, Sumio Morioka, Kohji Takano, and Seiji Munetoh. 2001. A Compact Rijndael Hardware Architecture with S-Box Optimization. In *Advances in Cryptology – ASIACRYPT 2001 (Lecture Notes in Computer Science, Vol. 2248)*. Springer, 239–254. doi:10.1007/3-540-45682-1_15

[21] Seeed Studio. 2024. PN532 NFC Library for Arduino. GitHub: https://github.com/Seeed-Studio/PN532. Arduino library for the NXP PN532 NFC controller.

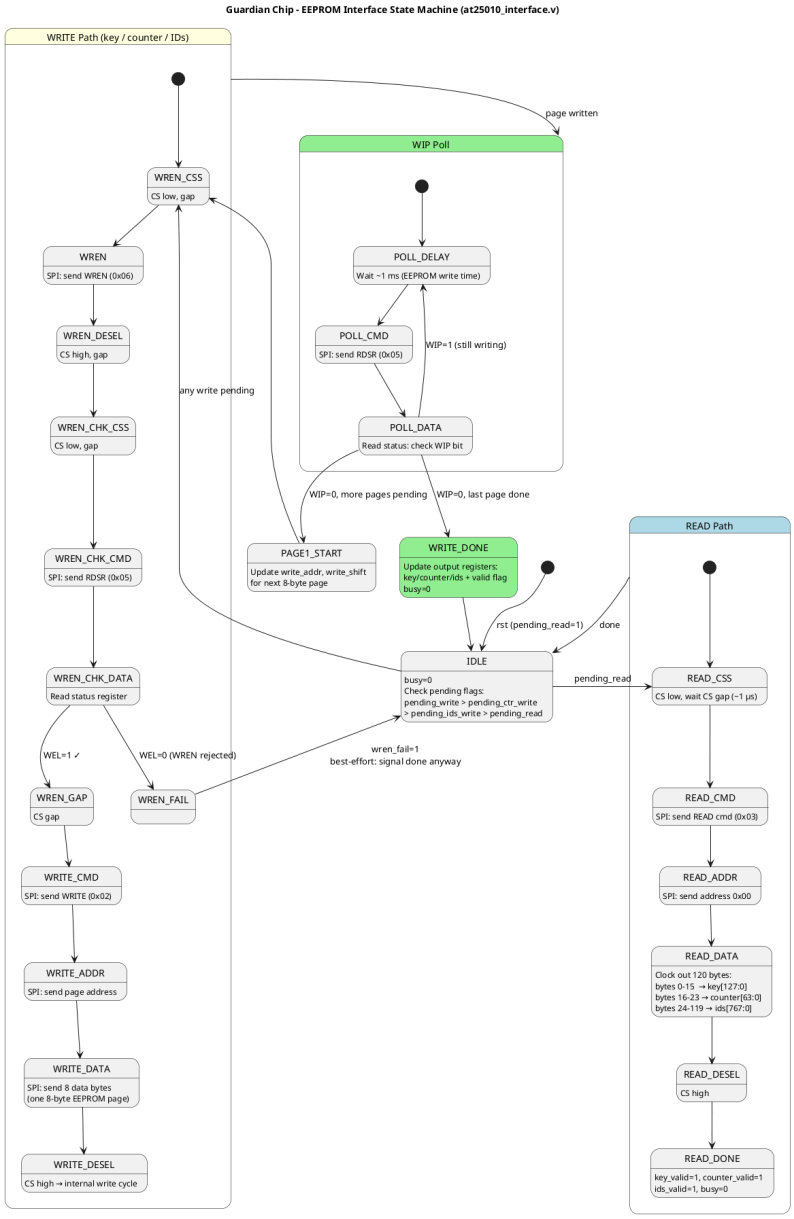

Fig. 2. State machine of the EEPROM interface (`at25010_interface.v`). A startup read is triggered automatically on reset. Write paths share the WREN/WEL verification and WIP polling logic.

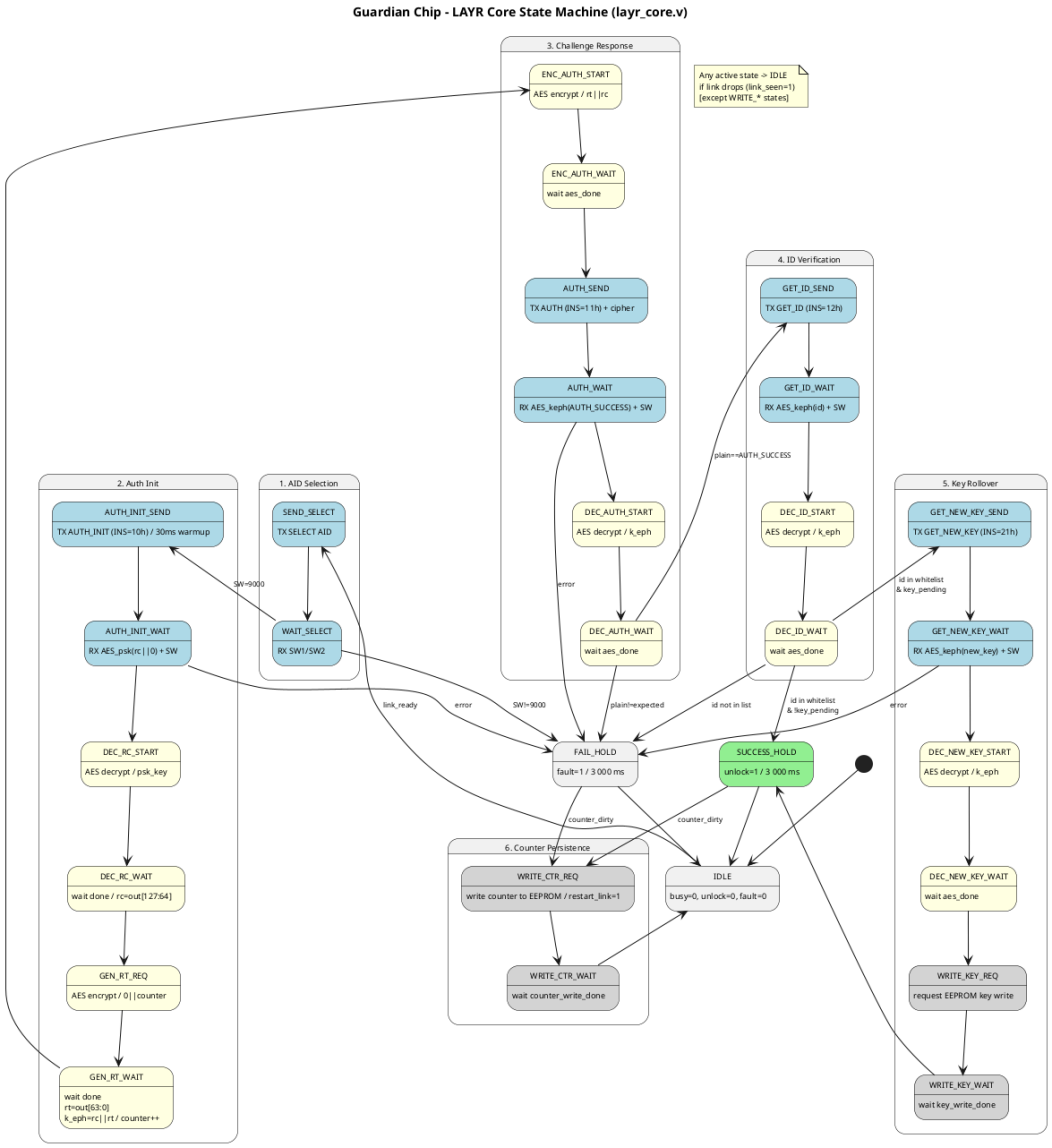

Fig. 3. State machine of the LAYR authentication core (`layr_core.v`). Yellow states invoke the AES engine; blue states exchange APDU bytes with the NFC interface; green states drive the unlock output. Any non-terminal state returns to IDLE if the NFC link drops.

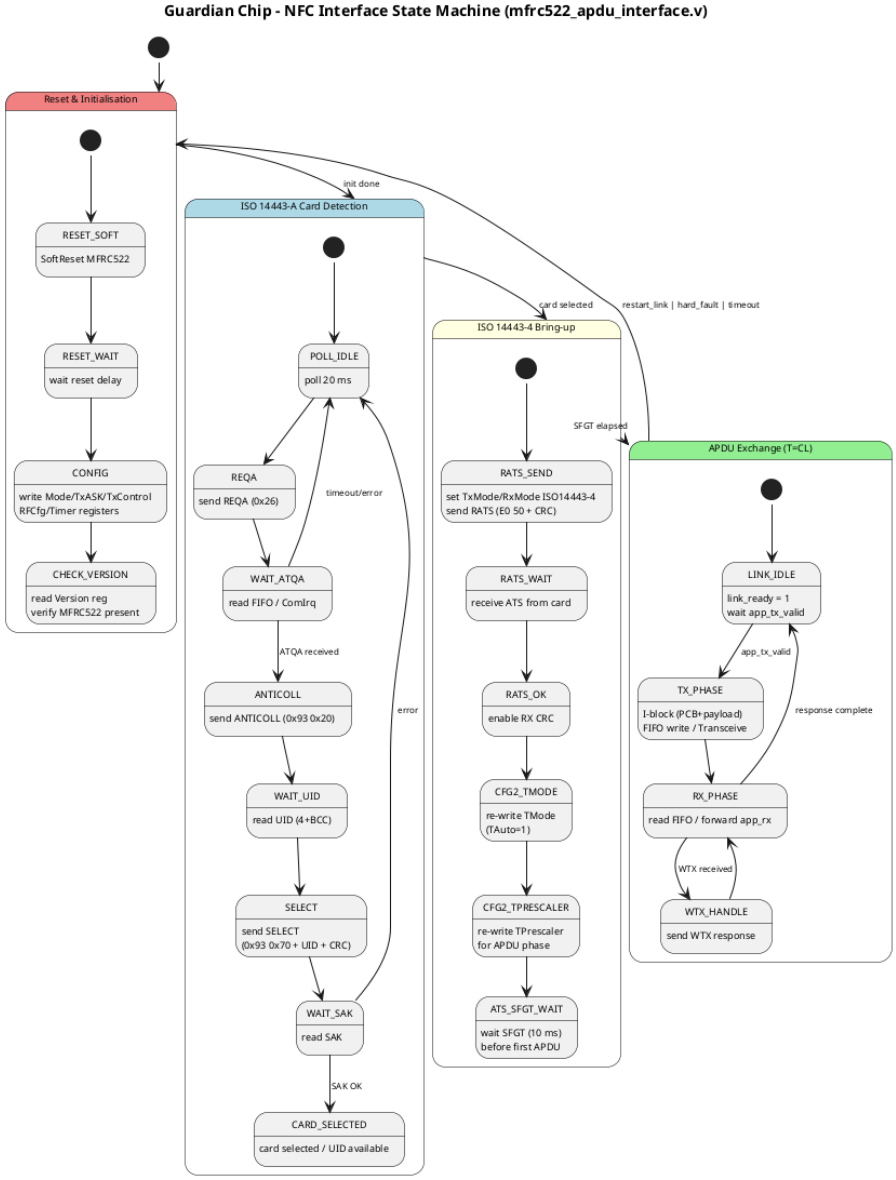

Fig. 7. State machine of the NFC interface module (`mfrc522_apdu_interface.v`). The four phase groups run left to right: hardware reset and configuration, ISO 14443-A card detection (REQA/ANTICOLL/SELECT), ISO 14443-4 link bring-up (RATS/ATS, timer reconfiguration, and SFGT wait), and transparent APDU I-block exchange including WTX handling.